I-BSIT-B GWA Calculator and Grade Tracker: The Complete Guide

By Carl Ianne, 1st-year BSIT · Nov 1, 2026 · @Carl

1. Overview

The I-BSIT-B GWA Calculator and Grade Tracker is a website where a student enters their scores, sees their current grade and GWA, and learns exactly what score they need on an upcoming exam. It was designed by Carl Ianne, a first-year BSIT student, and the code was written by Claude (Anthropic's AI assistant) from Carl's instructions. This guide explains what the program does, how it works, how it was built, and what has and has not been checked.

This guide assumes the site is deployed on Vercel. Claude did not deploy it and had no access to Vercel, so everything about the live site is written as the intended setup and is listed for checking in section 9.

What the program does

- Tracks courses, each split into components (Quizzes, Activities, Midterm, Finals, or any custom part) with the percent weight from the syllabus.
- Computes the current percent, the grade equivalent (for example 2.25) and the GWA across courses.
- Solves the what-if question: the score needed on the upcoming exam to pass or to reach a target grade.
- Saves everything in the browser's IndexedDB on the student's own device, and works offline as an installable app.
- Exports and imports Excel (.xlsx) and JSON files, so a student can restore their data if the browser's storage is wiped.
- Has a dark theme (dark gray and forest green) and a light theme (white with blue, red, yellow, orange and green).

How to read this guide

If you are...	Read these sections
A 1st-year BSIT student learning the code	5 (how it works), 6 (the key code), 11 (glossary), then the exercises in 10
A professor evaluating the work	2 (authorship), 3 and 4 (how it was built), 8 and 9 (what was tested and what was not), 10 (design decisions)
A classmate who just wants to use it	7 (features and how to use them)

2. Who did what (authorship and AI disclosure)

Claude, an AI assistant made by Anthropic, wrote all of the program's code and tests; Carl Ianne directed the work, made the design decisions, and reported what did not work. This is stated plainly so that anyone assessing the project knows what it does and does not show.

Area	Carl Ianne (1st-year BSIT)	Claude (AI assistant)
Idea and requirements	Defined what the tool must do: customizable course weights from the syllabus, scores by category, what-if calculator, themes, PWA, Excel import/export	Asked clarifying questions only where a wrong guess would be costly
Design decisions	Required saving in the browser and Excel re-import as a safety net; later chose IndexedDB, Vercel, offline use, no Flask, and Option A (use the SheetJS library file)	Explained the trade-offs and recommended static hosting
Code	Asked for modular files and beginner-friendly comments	Wrote every source file, comment, test and the flowchart
Testing	Used the app and reported a real bug: delete course did not delete	Wrote and ran the automated and browser tests (section 8) and fixed the bug

Area	Carl Ianne (1st-year BSIT)	Claude (AI assistant)
Deployment	Hosts the project on Vercel	Wrote the Vercel configuration and README steps; did not deploy
This guide	Requested it and set its audience	Wrote it from the project files and the conversation

What this means for assessment. The project shows a clear specification, sound design choices and honest testing. It does not, on its own, show that Carl wrote the code by hand. That is why section 10 includes exercises that check whether a student can read, change and explain the code.

Claude's models changed during the work (mostly Claude Sonnet 5.5, with short switches to other versions). The conversation shows each switch.

3. How it was built, step by step

The program was built twice in five days: first as a Flask (Python) app with local storage, then rebuilt as a static site with IndexedDB. Dates below are from the conversation (Asia/Shanghai time).

Version 1: Flask app with local storage (October 27 to 29, 2026)

1. **Requirements (Oct 27).** Carl described the tool: customizable components and weights, scores by category, a what-if solver, two themes, a PWA, Render hosting, saving in local web storage, and Excel export/import as a restorable save.
2. **Backend written.** Claude created a Flask app with an application factory and two blueprints, plus `models.py` (data shapes and cleaning), `grading.py` (the math) and `excel.py` (read and write .xlsx with openpyxl).
3. **Front-end written.** Themed CSS using color variables, then JavaScript modules for storage, math, dialogs and three screens (dashboard, course, settings).
4. **PWA parts.** A manifest, a service worker rendered by Flask with a list of files to cache, and generated icons.
5. **Testing and first fixes.** Unit tests and a real-browser run exposed three problems that were fixed: stray template-comment text printed at the top of every page, oversized icons in card titles, and a wrapping "All courses" link.

6. **Session limit (Oct 27).** The conversation hit its message limit. Carl asked Claude to check the existing files instead of starting over; Claude listed them, found the icons and tests already present, and fixed the leftover template bug.
7. **Delete-course bug (Oct 29).** Carl reported that deleting a course did nothing. Claude reproduced it in a browser, found the cause (section 6, code block 8), made a one-line fix and confirmed it.
8. **Flowchart (Oct 29).** The first diagram was tangled, so it was redrawn as five short left-to-right stories.

Version 2: static site with IndexedDB on Vercel (November 1, 2026)

9. **Decision.** Carl asked whether the app could keep every feature and its modular structure while moving to IndexedDB, Vercel, offline use and a new name. He then asked several questions, each time asking Claude to hold off on building anything, about what would change before approving. Claude explained that Flask must go (Vercel does not run it the same way), so Excel work moves into the browser.
10. **Excel library choice.** Carl chose Option A: use the SheetJS library file. Claude's sandbox could not download it (the network blocked every source), so the app was built to load it from `public/vendor/` and to show a clear message until it is added.
11. **Rebuild.** The Python logic was rewritten as JavaScript modules; storage moved to IndexedDB with a one-time copy from the old local storage; Excel became rows plus the library; `sw.js`, `vercel.json` and a dev server were added; the app was renamed.
12. **Testing.** 22 automated tests and a real-browser run (section 8). The count later became 30 after the review in the last step below. A phone-width problem was found and fixed: the "Saved" badge was cut off, so a shorter label is used on small screens.
13. **Documentation.** A new six-story flowchart, a README, and this guide.
14. **Code review and cleanup (Nov 2).** Carl asked for a check of whether the code was clean, readable and maintainable, with no spaghetti code or hidden logic problems. Claude reviewed it without changing anything and reported what it found. Carl then asked for all five fixes at once. They were: (1) the screens now refuse to add past the size limits and show a message, because a 21st component used to disappear silently on the next reload; (2) a custom target now uses the same rounding rule as the grade-scale rows (before, a custom 88% needed 98.58 on the final while the 88% row needed 97.15); (3) an unused variable and an out-of-date comment about the old Python version were removed; (4) the 340-line `course.js` was split into `course.js`, `what-`

`if-card.js` and `components-card.js`; (5) a new test checks the service worker's file list. While moving code, Claude also found and fixed an old bug: after adding or removing a component, the average labels went blank until the next edit.

Limits that shaped the build. The sandbox could not install npm packages (so no custom font) or gunicorn (so Version 1 was tested with Flask's built-in server). Neither changes how the final app works.

4. What Carl asked for (by Carl Ianne, 1st-year BSIT)

Carl gave 12 instructions across the project; this is the full log of what he asked for, summarised in plain words, with what each request led to.

#	Date	What Carl asked for	What it led to
1	Oct 27	A grade tracker where scores are logged by category, which works out the current standing and the score needed on the final to reach a target grade or just to pass. It had to be fully customizable from each syllabus, since this is something an automatic API cannot do	The component and weight model, the what-if solver, the per-syllabus customization
1 (cont.)	Oct 27	A dark default theme (dark gray, easy-on-the-eyes text, forest green) and a light theme using the five block colors (blue, red, yellow, orange, green), with a toggle	`theme.css` with two color sets and a toggle
1 (cont.)	Oct 27	Clean, modular files (models, routes and templates kept separate) with beginner-friendly comments so a first-year student can learn from them	The modular folder layout and the comment style in every file

#	Date	What Carl asked for	What it led to
1 (cont.)	Oct 27	Saving in local web storage, with the screen saying so, plus Excel export and import so a student can restore a save if their storage is wiped	Automatic saving, the visible "Saved" badge, Excel and JSON export/import
2	Oct 27	After a session limit, to continue the Flask project from the existing files instead of starting over: list what is there, then finish the icons and tests	File listing, the template-comment fix, a full re-run of tests
3	Oct 27	A casual question about how big this project is for a first-year student	A conversational answer about the scope of the project
4	Oct 29	A report that the delete-course button did nothing: the course stayed and could still be opened	Bug reproduced and fixed in `modal.js`
5	Oct 29	A flowchart picture inside the zip so a first-year student can understand how the program works	`docs/how-it-works-flowchart.png`
6	Nov 1	Whether every feature and the modular structure could be kept while moving to IndexedDB, Vercel hosting and offline use, with a new name for the app. Nothing was to be built yet	Feasibility answer; no files created yet
7	Nov 1	Whether dropping Flask would make the app do less when deployed. Nothing was to be built yet	Explanation that Flask's three jobs all move to the browser
8	Nov 1	A recap of the plan before any building	An 11-step plan
9	Nov 1	A plain-words explanation of the Excel library choice	A plain explanation of Option A and Option B

#	Date	What Carl asked for	What it led to
10	Nov 1	Option A chosen, with the plan to be confirmed once more first	A confirmed plan with Option A
11	Nov 1	Approval to start building	The static IndexedDB version (Version 2)
12	Nov 1	A dedicated guide for first-year BSIT students and professors covering the program, how it works, the key code, how it was built, and what was and was not tested	This guide

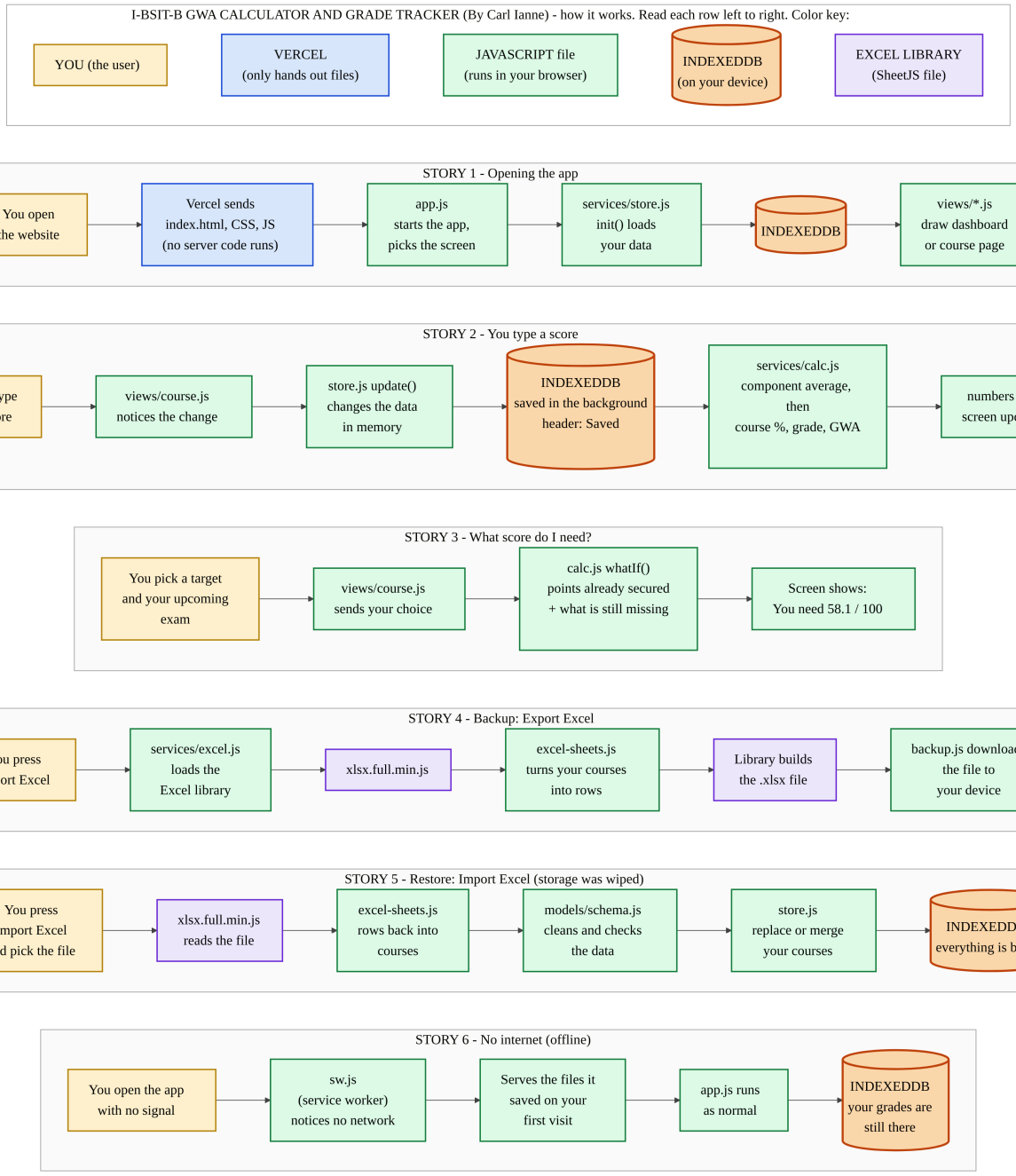
Requests after this guide was first written (Nov 1–2, the code review)

#	Date	What Carl asked for	What it led to
13	Nov 2	A read-only check of whether the code is clean, readable and maintainable, and free of tangled logic or hidden problems. Nothing was to be changed yet	A review: the structure is sound; it found the silent size-limit data loss, the custom-target rounding mismatch, dead code, a stale comment, and an oversized `course.js`
14	Nov 2	Confirmation of the five proposed fixes before any changes	The plan restated file by file; nothing changed
15	Nov 2	Whether all of it could be done in one go, with a fresh zip sent back. Nothing was to be changed yet	Confirmed that was possible; nothing changed
16	Nov 2	A check that Claude understood this would also mean updating this guide	An explanation of which guide sections would be updated, and which would not
17	Nov 2	Approval to run the whole sequence	The five fixes, new and updated tests (30 total), a repeat of the browser run, the updates to this guide, and a new zip

What the requests show. Carl set requirements that were specific and testable, asked for explanations before approving big changes, and reported a bug found by actually using the app. Several of the later messages are decisions rather than requests, and those decisions (IndexedDB, Vercel, offline, Option A) shaped the final architecture.

5. The big picture: how the program fits together

The flowchart below is also in the project zip as `docs/how-it-works-flowchart.png`. It follows six short stories, such as opening the app, typing a score, and going offline.



The idea in one paragraph. The page (`index.html`) loads `app.js` . The app keeps one copy of all your data in memory (the store). When you change something, the store updates that copy, tells the screen to redraw, and saves the copy to IndexedDB a moment later. The grade math lives in its own file and never touches the screen or the database.

Folder map

Folder or file	Job
`public/index.html`	The single page and its header
`public/js/app.js`	Starts everything and switches between screens
`public/js/config.js`	App name and storage names in one place
`public/js/models/`	`schema.js` (data shape and `normalize`) and `presets.js` (ready-made scales and course templates)
`public/js/services/`	The logic: `db.js` (IndexedDB), `store.js` (state and saving), `calc.js` (grade math), `excel-sheets.js` , `excel.js` , `backup.js`
`public/js/ui/`	Small helpers: `dom.js` , `format.js` , `icons.js` , `modal.js` , `toast.js`
`public/js/views/`	One file per screen: `dashboard.js` , `course.js` (page frame and the standing card), `what-if-card.js` , `components-card.js` , `course-form.js` , `settings.js`
`public/sw.js` , `manifest.webmanifest`	Offline support and installability
`public/css/`	`theme.css` (colors), `base.css` , `components.css`
`public/vendor/`	Where the SheetJS file must be placed
`tests/`	The 30 automated tests
`vercel.json`	Hosting settings and security headers

Rule of thumb for reading the code: start with `calc.js` (pure math), then `schema.js` , then `store.js` , and only then the views. Each layer only uses the ones before it.

6. The essential code, explained

Nine pieces of code carry the whole program; each is quoted below from the final project (Version 2) with a plain explanation. All paths are inside the `public/` folder.

Code block 1: cleaning data (`js/models/schema.js`)

Every number that enters the app passes through this function first, because data from a file or a text box cannot be trusted.

```
export function toNum(value, fallback = null) {
  if (value === null || value === undefined || value === "" || typeof value
=== "boolean") return fallback;
  const n = typeof value === "number" ? value :
parseFloat(String(value).replace(/,/g, ""));
  return Number.isFinite(n) ? n : fallback;
}
```

It turns "12,5", " 7 " or 7 into a real number and returns a safe fallback for anything else. `normalize()` in the same file applies it to a whole save, for example `score: toNum(i.score) === null ? null : Math.max(0, toNum(i.score))` (an empty score stays empty, a negative score becomes 0) and `max: Math.max(0.0001, toNum(i.max, 100))` (a maximum can never be 0, which would divide by zero later).

Code block 2: a component's average and a course's standing (`js/services/calc.js`)

```
export function componentStats(component) {
  let score = 0, max = 0;
  for (const item of component.items) {
    if (item.score !== null && item.score !== undefined && item.max > 0) {
      score += item.score;
      max += item.max;
    }
  }
  return { score, max, pct: max > 0 ? (score / max) * 100 : null };
}

export function courseStanding(course, settings) {
  const totalWeight = course.components.reduce((sum, c) => sum + c.weight,
0);
  let gradedWeight = 0, earned = 0;
```

```
  for (const comp of course.components) {
    const { pct } = componentStats(comp);
    if (pct !== null) {
      gradedWeight += comp.weight;
      earned += (comp.weight * pct) / 100;
    }
  }
  const percent = gradedWeight > 0 ? (earned / gradedWeight) * 100 : null;
  // ...grade, passed and the totals are returned too
}
```

Two rules sit in this code. A component's average is total points earned divided by total points possible (not the average of each item's percent), and items without a score are skipped, so upcoming work does not count as zero. The course percent only uses components that have at least one score, as the formula shows:

```
\text{course \%} = \frac{\sum_i w_i \, p_i}{\sum_i w_i}
```

Here w is a component's weight and p its average percent, summed over graded components only. The GWA is then the unit-weighted average of each course's grade equivalent.

Code block 3: the what-if solver (`js/services/calc.js`)

```
const share = (comp.weight / total) * 100;          // this component's % of
the course
// ...secured = course-% points already earned in the OTHER components
const have = componentStats(comp);                  // scores already inside
this component
const needAvg = ((target - secured) / share) * 100; // average this component
must reach
const needed = (needAvg / 100) * (have.max + newMax) - have.score;
```

In words: take the target, subtract what the other components already secured, divide by how much the chosen component is worth, then convert that average into a score on the upcoming item. In symbols, with T the target, S the secured points, s the component's share, E and M the points already earned and possible in it, and N the maximum of the upcoming item:

```
x = \frac{T - S}{s}\,(M + N) - E
```

Worked example (this is unit test 5 in `calc.test.mjs`): Quizzes are 8/10 and worth 40%, so S = 0.40 × 80 = 32. The Final is worth 60% and T = 75, so x = (75 − 32) / 60 × 100 = 71.67 out of 100. The code then labels the result `secured` (a score of 0 is enough), `impossible` (even a perfect score falls short) or `needed`.

Code block 4: saving without freezing (`js/services/store.js`)

```
export function update(mutator) {
  mutator(state);
  save();
}

export function save() {
  dirty = true;
  if (!writing) writing = flush();
  return writing;
}

async function flush() {
  let result;
  try {
    while (dirty) {
      dirty = false;
      await db.put("state", structuredClone(state));
    }
    storageWorks = true;
    result = { ok: true, at: new Date() };
  } catch (error) {
    dirty = false;
    storageWorks = false;
    result = { ok: false, error };
  }
  writing = null;
  emitSaved(result);
  if (result.ok) channel?.postMessage("changed");
}
```

The screens read from `state`, an ordinary object in memory, so they are instant. `update()` changes it and calls `save()`, which writes a copy to IndexedDB in the background. If you type five scores quickly, `dirty` makes the loop write the latest copy

again instead of starting five separate writes (the store test about merged writes checks this). `structuredClone` saves a snapshot so a later edit cannot change data mid-write, and the `catch` turns a failed write into a visible "Not saved" badge instead of a crash.

Code block 5: talking to IndexedDB (`js/services/db.js`)

```
async put(key, value) {
  const db = await openDatabase();
  return new Promise((resolve, reject) => {
    const tx = db.transaction(STORE, "readwrite");
    tx.objectStore(STORE).put(value, key);
    tx.oncomplete = () => resolve();
    tx.onerror = () => reject(tx.error);
    tx.onabort = () => reject(tx.error);
  });
}
```

IndexedDB reports results through callbacks. This function wraps one write in a Promise so the rest of the code can simply write `await db.put("state", data)`. The database has one table called `kv` (key to value) holding the whole save under the key `state`.

Code block 6: Excel rows back into courses (`js/services/excel-sheets.js`)

```
const courseKey = `${text(courseName).toLowerCase()}|${text(code, "",
30).toLowerCase()}`;
if (!courses.has(courseKey)) {
  courses.set(courseKey, { name: text(courseName), code: text(code, "", 30),
units: toNum(units, 3), components: [] });
}
const course = courses.get(courseKey);
```

The Excel "Data" sheet has one row per score, and the course name repeats on every row. This code uses a `Map` keyed by course name and code to group those rows back into one course each, then does the same for components. Because the layout never changed between versions, files exported by the old Flask version still import (test 12 uses a real file from it).

Code block 7: working offline (`sw.js`)

```
self.addEventListener("fetch", (event) => {
  const request = event.request;
  const url = new URL(request.url);
  if (request.method !== "GET" || url.origin !== self.location.origin)
return;
  event.respondWith((async () => {
    const cache = await caches.open(CACHE);
    try {
      const response = await fetchWithTimeout(request, 4000);
      if (response.ok) cache.put(request, response.clone());
      return response;
    } catch {
      const saved = await cache.match(request, { ignoreSearch: true });
      if (saved) return saved;
      if (request.mode === "navigate") return (await
cache.match("index.html")) || Response.error();
      return Response.error();
    }
  })());
});
```

The service worker sits between the page and the network. Online, it fetches the newest file and refreshes its saved copy, so a new deployment appears right away. If the network fails or takes more than 4 seconds, it serves the saved copy, which is why the app opens with no signal.

Code block 8: the delete-course bug and its fix (`js/ui/modal.js`)

```
// Esc key = cancel. The browser fires "close" a moment AFTER dialog.close(),
so when one dialog
// hands over to another (Edit course -> "Are you sure?"), that late event
arrives while the NEW
// dialog is already open. Ignoring it when the dialog is open stops it from
cancelling the new one.
dialog.onclose = () => { if (!dialog.open) finish(null); };
```

The bug: pressing "Delete course" closed the edit dialog and opened a confirmation dialog, but the first dialog's late "close" event reached the second one and counted as "Cancel". The fix is the added condition `if (!dialog.open)`. It was found because Carl

used the app, reproduced in an automated browser run, and confirmed fixed by the same run (the course count in IndexedDB went from 1 to 0).

Code block 9: building the screen safely (`js/ui/dom.js`)

```
function append(parent, children) {
  for (const child of children) {
    if (Array.isArray(child)) append(parent, child);
    else if (child === null || child === undefined || child === false)
continue;
    else parent.append(child instanceof Node ? child :
document.createTextNode(String(child)));
  }
}
```

Every screen is built with a small helper `h(tag, props, ...children)`. Text a user types is always added as a text node, never as HTML, so a course named `<script>` is displayed literally and cannot run code. This is the main defense against a class of attacks called cross-site scripting.

7. Features and how to use them

A student sets up a course, enters scores, and sees their standing on one screen. Everything below works with no internet after the first visit; Excel export and import also need the SheetJS library file to be in place (section 9).

Getting started

1. Open the site and choose **Add course** (or **Try with sample data** to see an example).
2. Pick a template (Typical, Lecture + Lab, or Blank), then change each component's name and weight to match your syllabus. The page shows whether the weights add up to 100%.
3. Under each component choose **Add item** and enter each quiz, activity or exam as a score and a maximum. Press Enter in a score box to jump to the next one. Leave a score empty for work that has not happened yet.
4. Watch **Current standing** update: current percent, grade equivalent, passing or not, and how much of the course is graded.

The what-if calculator (on each course page)

1. Under **I want to get**, choose "Just pass", any grade from your scale, or a custom percent.
2. Under **On my upcoming**, choose the component still to come (usually the Final Exam) and its maximum points.
3. Read the result: the score needed, "You already have it!", or "Out of reach". The table underneath shows the score needed for every grade at once.

Settings

- **Grading rules:** the passing percent, the grade given below the scale, and whether percents are rounded to a whole number first.
- **Grade scale:** edit each row (from percent, grade) or load a preset (Philippines 1.00 to 5.00, or US 4.0). The default Philippine scale is a common one and must be checked against your school's own table.
- **Backup and restore:** Export Excel, Import Excel, Download JSON, Restore JSON. Importing offers "Add to existing" or "Replace everything".
- **Delete all data:** erases the courses stored on this device (after a confirmation).

Install and offline use. On a phone, open the site and choose Add to Home Screen (or Install app). After the first visit the app opens with no signal, and the header badge shows "Saved" each time your data is written to IndexedDB.

Keep a backup. Browser storage can be erased by clearing site data, using a private window, or changing phones. Export an Excel or JSON file after you enter your scores, and keep it somewhere safe.

8. Verification: what was tested and how

The final version passed all 30 automated tests and a real-browser run covering saving, restoring, deleting, themes, phone layout and offline use. The browser run used Chromium (driven by Playwright) against a local server that applies the same security headers as `vercel.json`. Nothing in this section was tested on the live Vercel site (section 9).

Automated tests (run with `npm test`; 30 of 30 passed)

File	Tests	What they prove
`calc.test.mjs`	8	Component averages skip ungraded items; course percent and grade; rounding (74.4 fails, 74.5 passes at 75); unit-weighted GWA; what-if needs 71.67 on the Final in the worked example; unreachable and already-secured targets; what-if inside a component that has scores; the rounding rule shared by scale rows and custom targets
`core.test.mjs`	8	Messy input is cleaned (negative score becomes 0, weight above 100 becomes 100, maximum above 0); non-saves are rejected and huge input is capped at 60 courses; Excel rows round-trip; a name like `=cmd()` stays text; a real export from the old Flask version still imports; friendly import errors; the library glue round-trips and reports unreadable files (using a stand-in for SheetJS)
`limits.test.mjs`	4	The limit guard stays silent while there is room and refuses with a message at the limit; a save exactly at every limit survives a reload; one over the limit is trimmed (the reason the screens must refuse it)
`store.test.mjs`	7	Empty start; saves reach the database and quick edits are merged; data survives a restart; the old local-storage save is copied in once; damaged data is kept as a copy instead of deleted; a failing disk shows "not saved" instead of crashing; junk is rejected on restore
`sw.test.mjs`	3	Every file in the service worker's list exists; every JavaScript and CSS file is in the list; no duplicates. Checked by removing one entry on purpose: the test failed, then passed after the entry was restored

Real-browser checks (Version 2)

Check	Result
Page title and header show the new name	Passed

Check	Result
Typing a score writes it to IndexedDB; the badge shows "Saved on this device (IndexedDB)"	Passed
Reloading the page shows the saved score; no app data is kept in localStorage	Passed
Excel button with the library file missing shows the friendly message	Passed
JSON backup downloads; Delete all data empties IndexedDB; Restore JSON brings the course back (GWA 2.25)	Passed
Delete course (the earlier bug) removes the course (1 to 0 in IndexedDB)	Passed
Theme toggle switches to light mode and the choice is remembered	Passed
An old local-storage save is copied into IndexedDB with a notice	Passed
With the network switched off, a reload still shows the app and the saved course; 32 files were cached (29 before the split added three)	Passed
Phone width (390 px): no sideways scrolling; short "Saved · IndexedDB" badge fits	Passed
After the review fixes: the whole run above repeated on the split code, with the same results	Passed
At 20 components, 200 items in one component, and 60 courses, the Add buttons show a limit message and add nothing; the counts are unchanged after a reload	Passed
A custom target equals the matching scale row (rows 97, 94, 91 and 88 compared)	Passed. In the old version the 88% case differed (98.58 vs 97.15)

Check	Result
Average labels after adding a component	Old version: 3 labels before, 0 after (a bug). New version: 3 before, 3 after
Browser console during the run	Only one error: the expected 404 for the missing Excel library file

Checks done on Version 1 (Flask)

- 10 Python tests passed (math, a real openpyxl Excel round-trip, formula-injection protection, routes).
- A browser run confirmed saving, reload, theme switch, Excel export, wipe-and-import restore, phone layout and that typing a score then pressing Tab keeps the cursor in place.
- The delete-course bug was reproduced before the fix and confirmed fixed after it.

Method. Claude ran each test itself and looked at screenshots of the dark, light and phone layouts. Where a screenshot showed a problem (stray text, huge icons, a cut-off badge), it was fixed and the run was repeated.

9. What was not tested, known limits, and the after-deploy checklist

This section is deliberately blunt. If the app is live on Vercel when you read this, the items below are still unconfirmed until someone completes the checklist.

Not tested

- **Excel export and import with the real SheetJS library.** The library file could not be downloaded in the build environment, so only a stand-in was tested. The code that groups rows, protects text cells and reads the file is covered; the real library is not.
- **The live Vercel deployment.** Claude cannot deploy. The security headers in `vercel.json` were only imitated by the local test server, so the real response headers are unconfirmed.
- **Installing as an app** (Add to Home Screen on Android, Install in Chrome or Edge, Share menu on iPhone). The manifest and icons exist, but no real phone or install prompt was used.
- **Browsers other than Chromium.** Firefox and Safari were not run. Safari in particular may clear IndexedDB for sites not used for about a week.

- **Cross-tab sync** (editing in two tabs at once) is built on BroadcastChannel but was not exercised in a browser.
- **Updating a cached version.** The service worker uses a fixed cache name (`ibsit-b-gwa-v1`); the update behaviour after a new deploy was not tested.
- **Real users, screen readers and long-term use** such as hundreds of scores or a full semester of data.

Known limits

- Data lives only in this browser on this device. It is not shared between devices and is lost if the browser data is cleared. Download a JSON or Excel backup regularly.
- Caps: 60 courses, 20 components per course, 200 items per component, 40 grade-scale rows. The screens stop at these limits and show a message. (Before the Nov 2 review, going over a limit looked fine until the next reload, when the extra item vanished.) Importing a save that would push you over 60 courses is refused with a message.
- The default grade scale is a common Philippine-style scale; your school's may differ. Edit it in Settings.
- The what-if answer assumes the remaining items count equally by their maximum scores.
- This is a personal planning tool, not an official record.

After-deploy checklist (11 steps, in order**)**

1. Download SheetJS 0.20.3 (`xlsx.full.min.js`) from the official SheetJS site and place it in `public/vendor/`. The link given earlier was not verified, so check the version on their download page.
2. Run `npm test` (30 tests should pass) and `npm run dev`, then open `http://localhost:8000`.
3. Push to GitHub and import the repo into Vercel. No build command is needed; the output folder is `public` (set in `vercel.json`).
4. Open the live URL and enter one course. Confirm the badge says "Saved on this device (IndexedDB)".
5. Reload and confirm the score is still there.
6. Click Export Excel, open the file, then wipe all data and use Import Excel to bring it back.
7. In DevTools, open Network and check the page response headers include the Content-Security-Policy.

8. Switch the network to Offline and reload. The app should still open.
9. On a phone, install it (Add to Home Screen) and open it from the icon.
10. Test on Firefox and Safari if students use them.
11. When you change the app later, change the cache name (for example `v2`) so installed copies refresh.

Tick each step only when you have seen it work. Section 8 records what Claude saw; this list records what you still need to see.

10. For professors: design decisions and exercises

Design decisions and why

Decision	Reason	Trade-off
Dropped Flask; static site	Everything the app does runs in the browser, so no server is needed. Vercel serves the files and offline use works	Data cannot be shared between devices
IndexedDB instead of localStorage	Larger, structured storage that is more robust	Slightly more code (`db.js`)
Plain JavaScript modules, no framework	A first-year student can read every file from top to bottom	Manual screen updates
Grade math in `calc.js` with no browser code	It can be tested by Node alone, and the 7 calc tests prove it	None worth noting
`normalize()` at every entry point	Every save, import and restore passes the same cleaning step, so bad data cannot break screens	Extra code in `schema.js`
Text nodes, not `innerHTML`	A course named `<script>` shows as plain text	Longer helper code in `dom.js`
SheetJS loaded only when needed	The app starts fast and works without it	The user must add one file
Network-first service worker with a 4 s timeout	Updates show up when online and the app still opens offline	A slow connection waits up to 4 s

What the build shows about AI-assisted work

- Claude wrote and ran the tests, but Carl read and challenged the results. Carl's report of the delete bug is the clearest example: a feature that looked finished was broken in real use, and the cause (a late `close` event) was found only after reproducing it.
- Several of Claude's early statements were corrected before the guide was finished (timings and counts that could not be backed up).
- A real library file is still missing, which is why section 9 exists.

Discussion questions

1. Why does `componentStats` skip ungraded items instead of counting them as zero? What would the student see if it did not?

2. In the what-if formula, what do `T`, `S`, `s`, `M`, `N` and `E` mean, and why can the answer be above 100?
3. What are the risks of keeping a single copy of student data in one browser? How do the JSON and Excel backups reduce them?
4. Why does the app write the in-memory state first and the database second? What could go wrong the other way around?

Student exercises

1. *Easy:* Change the default `passingPercent` (75) in `schema.js` and rerun `npm test`. Predict first which tests, if any, will fail, then check.
2. *Easy:* Add a new entry to `SCALE_PRESETS` in `presets.js` and apply it from Settings.
3. *Medium:* Add a test to `calc.test.mjs` for a course with no components. Predict the result before running it.
4. *Medium:* Raise the course cap in `schema.js` and update the matching test.
5. *Harder:* Add an "Honors" label on the dashboard when the GWA is at or below a threshold.
6. *Harder:* Bump the cache name in `sw.js`, redeploy, and describe what a returning visitor sees.

11. Glossary

Term	Plain meaning
GWA	General Weighted Average: each course's grade multiplied by its units, added up, divided by total units
Component	A graded part of a course, such as Quizzes or Final Exam, with a percent weight
Item	One scored entry inside a component, such as Quiz 1: 18 out of 20
What-if	A calculation that asks "what do I need on the remaining work to reach my target?"
PWA	Progressive Web App: a website that can be installed and opened like an app

Term	Plain meaning
Service worker	A script the browser runs in the background that can answer page requests from saved copies, which is what makes offline use work
IndexedDB	A database built into the browser that stores structured data on the user's device
localStorage	A smaller, simpler browser storage for text values; this app now uses it only for the theme choice
ES module	A JavaScript file that shares code with `import` and `export`
Normalize	Cleaning data into a safe, expected shape before the app uses it
CSP	Content Security Policy: a header telling the browser which sources a page may load code from
Static site	A site made only of files that are sent to the browser as they are, with no server code
Vercel	A hosting service that serves a project's files from the internet
SheetJS	A JavaScript library that reads and writes Excel files in the browser
BroadcastChannel	A browser feature that lets tabs of the same site send each other messages
Write-through save	Changing the in-memory copy first, then saving it to the database right after